

Module-4 Notes

Document Databases

Document databases are a type of NoSQL database that store, retrieve, and manage data in the form of documents, typically using formats like JSON, BSON, or XML. Unlike relational databases that store data in tables with predefined schemas, document databases are schema-less or have flexible schemas, allowing for the storage of varied data structures in the same collection. Each document in a document database is a self-contained unit of data, and documents can contain nested structures, such as arrays or sub-documents, making them highly flexible and suitable for hierarchical or complex data. Popular document databases like MongoDB and CouchDB enable fast querying, indexing, and scalability, and they support features like full-text search, aggregation, and secondary indexing. Because of their flexible schema and ability to scale horizontally across distributed systems, document databases are ideal for applications with rapidly changing data structures, such as content management systems, e-commerce platforms, and real-time analytics. Eg:

Oracle	MongoDB
database instance	MongoDB instance
schema	database
table	collection
row	document
rowid	_id
join	DBRef

Features of Document oriented databases

Document-oriented databases offer several key features that make them suitable for flexible and scalable data management in modern applications. Here are some of the important features:

- ✚ Schema Flexibility: Document databases do not require a predefined schema, allowing for dynamic and flexible data storage. Each document can have its own structure, with varying fields and data types, making it easier to handle changing data models.
- ✚ JSON/BSON/XML Storage: Documents are typically stored in formats like JSON, BSON, or XML, which are human-readable and allow for complex, nested data structures. This makes it easy to represent hierarchical data and reduces the need for normalization, as opposed to relational databases.
- ✚ Indexing and Querying: Document databases support powerful indexing mechanisms, enabling efficient searches on various fields, including those within nested structures. Complex queries, including range queries and full-text search, can be executed quickly.
- ✚ Scalability: Document databases are designed to scale horizontally by distributing data across multiple servers or nodes. This makes them suitable for handling large volumes of data and high traffic, especially in distributed systems.
- ✚ Data Aggregation: Many document databases, such as MongoDB, provide built-in aggregation frameworks that allow for complex data analysis and transformation. This is useful for analytics and reporting without needing to export data to an external processing system.

- ✚ High Availability and Fault Tolerance: Document databases typically support replication and automatic failover, ensuring high availability and data durability. Data is replicated across multiple nodes to provide fault tolerance, so the system continues to function even in the event of hardware failures.
- ✚ Atomic Operations: Some document databases support atomic operations, allowing modifications to be made to individual documents in a way that ensures consistency. For example, MongoDB supports atomic updates on individual documents, providing reliable data integrity.
- ✚ Ease of Integration: Due to their flexible schema and use of JSON-like data formats, document databases are well-suited for integration with web applications, APIs, and microservices, making them a popular choice for modern software architectures.

MongoDB

MongoDB is a popular, open-source, document-oriented NoSQL database designed for scalability, flexibility, and high performance. It stores data in a JSON-like format called BSON (Binary JSON), which allows for complex, nested structures that can evolve over time without requiring a fixed schema. MongoDB supports horizontal scaling through sharding, where data is distributed across multiple servers, ensuring high availability and the ability to handle large datasets and high-traffic applications. Its powerful query language allows for efficient data retrieval, including support for full-text search, aggregation, and geospatial queries. MongoDB also provides features like automatic failover, replica sets for data redundancy, and ACID-compliant transactions for reliable data consistency. Because of its flexibility and scalability, MongoDB is widely used in applications ranging from real-time analytics and content management systems to e-commerce platforms and mobile applications.

CAP Theorem and MongoDB

The CAP Theorem (Consistency, Availability, Partition Tolerance) states that a distributed system can only guarantee two of the three properties at the same time: Consistency (every read operation returns the most recent write), Availability (every request will receive a response, even if it's not the most up-to-date data), and Partition Tolerance (the system continues to operate despite network or hardware failures that split the system into partitions). MongoDB, being a distributed database, primarily adheres to the principles of the CA (Consistency and Availability) model under normal conditions, ensuring that all nodes in a replica set are consistent with each other and that the system remains available for read/write operations. However, MongoDB can be configured to prioritize Partition Tolerance over consistency in situations where network partitions occur. By default, MongoDB uses eventual consistency, which can lead to temporary inconsistencies across replica sets until data is synchronized, making it better suited for situations where availability and partition tolerance are more critical than strict consistency, especially in distributed systems with high traffic.

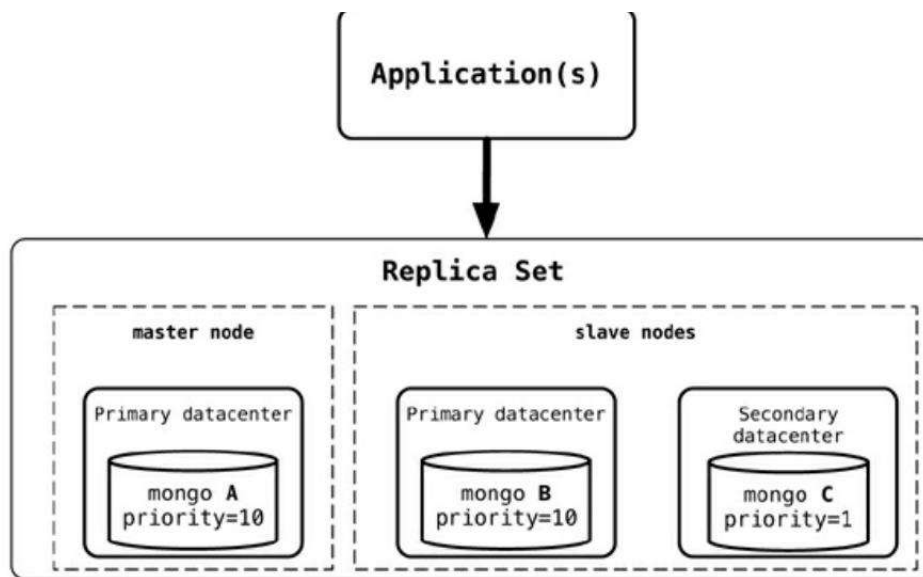
Availability in MongoDB

In MongoDB, replica sets are a group of MongoDB servers that maintain the same data set, providing redundancy and high availability. A replica set consists of a primary node and one or more secondary nodes. The primary node handles all write operations, while the secondary nodes replicate the data from the primary. If the primary node fails, one of the secondary nodes can be automatically promoted to become the new primary, ensuring that the database remains available and minimizing downtime.

Key Features of Replica Sets:

- ✚ Automatic Failover: If the primary node becomes unavailable, the replica set will automatically elect a new primary from the secondary nodes to ensure continuous availability.
- ✚ Data Replication: Data is replicated from the primary to the secondaries asynchronously, meaning the secondaries may lag slightly behind the primary. This provides fault tolerance and ensures that there are multiple copies of the data across different nodes.
- ✚ Read and Write Distribution: By default, writes are directed to the primary node, while reads can be directed to either the primary or secondary nodes. However, this can be configured to prioritize reading from the primary or only from secondaries.
- ✚ Consistency and Durability: MongoDB supports strong consistency for write operations on the primary, while reads can be configured for consistency or availability depending on the application's needs. Replica sets help ensure data durability and protection against data loss in case of server failures.
- ✚ Configurable Write Concern and Read Concern: MongoDB allows fine-grained control over how many nodes must acknowledge a write or read operation, offering a balance between consistency, availability, and performance.

Eg:



Replica set configuration with higher priority assigned to nodes in the same datacenter

The application writes or reads from the primary (master) node. When connection is established, the application only needs to connect to one node (primary or not, does not matter) in the replica set, and the rest of the nodes are discovered automatically. When the primary node goes down, the driver talks to the new primary elected by the replica set

MongoDB CRUD operations

//To list all databases

```
show dbs;
```

//To create or use databases

```
use blog;
```

//Creating a collection (table) inside databases

```
db.createCollection("posts");
```

```
//Inserting documents (Create)
```

```
db.posts.insertOne({
  title: "Post Title 1",
  body: "Body of post.",
  category: "News",
  likes: 1,
  tags: ["news", "events"],
  date: Date()
})
```

```
db.posts.insertMany([
  {
    title: "Post Title 2",
    body: "Body of post.",
    category: "Event",
    likes: 2,
    tags: ["news", "events"],
    date: Date()
  },
  {
    title: "Post Title 3",
    body: "Body of post.",
    category: "Technology",
    likes: 3,
    tags: ["news", "events"],
    date: Date()
  },
  {
    title: "Post Title 4",
    body: "Body of post.",
    category: "Event",
    likes: 4,
    tags: ["news", "events"],
    date: Date()
  }
])
```

//Retrieval operations

```
db.posts.find()
db.posts.findOne()
db.posts.find( {category: "News"} )
```

```
db.posts.find({}, {title: 1, date: 1})
db.posts.find({}, {_id: 0, title: 1, date: 1})
db.posts.find({}, {category: 0})
```

//Update operations

```
db.posts.updateOne( { title: "Post Title 1" }, { $set: { likes: 2 } })
db.posts.updateOne(
  { title: "Post Title 5" },
  {
    $set:
    {
      title: "Post Title 5",
      body: "Body of post.",
      category: "Event",
      likes: 5,
      tags: ["news", "events"],
      date: Date()
    }
  },
  { upsert: true }
)

db.posts.updateMany({}, { $inc: { likes: 1 } })
```

//Delete operations

```
db.posts.deleteOne({ title: "Post Title 5" })
db.posts.deleteMany({ category: "Technology" })
```

Module-4 (Contd..)

Availability

MongoDB offers several options to ensure high availability:

1. Replica Sets:
 - Primary and Secondary Nodes: In a replica set, data is replicated across multiple nodes. One node is the primary that receives all write operations, while secondary nodes replicate the data from the primary and can serve read operations.
 - Automatic Failover: If the primary node goes down, an eligible secondary node is automatically promoted to primary, ensuring that the database remains available.
2. Sharding:

- Data Distribution: Sharding distributes data across multiple servers or clusters, which can help in scaling out horizontally. Each shard is a replica set, ensuring data redundancy and high availability.
- Query Routing: MongoDB uses a query router (mongos) to direct operations to the correct shard, which helps maintain performance and availability.
- 3. Cloud Deployment:
 - MongoDB Atlas: A fully managed cloud database service that offers automated backups, monitoring, and alerts. Atlas ensures high availability through built-in replication and failover mechanisms across multiple regions and availability zones.
- 4. Backup and Recovery:
 - Regular Backups: Regularly scheduled backups and point-in-time recovery options help in maintaining data integrity and availability in case of failures.
 - Snapshot and Continuous Backup: Cloud providers like MongoDB Atlas offer continuous backups and snapshot capabilities, enabling quick recovery from data loss incidents.
- 5. Geographical Distribution:
 - Multi-Region Deployment: Deploying replica sets across different geographic regions can protect against regional outages, ensuring that your database remains available globally.
- 6. Maintenance and Upgrades:
 - Rolling Upgrades: MongoDB supports rolling upgrades, allowing you to update the database software without significant downtime.

Replica-sets

A MongoDB replica set is a group of MongoDB instances that maintain the same data set, providing redundancy and high availability. Here's a detailed overview of how replica sets work:

Components of a Replica Set

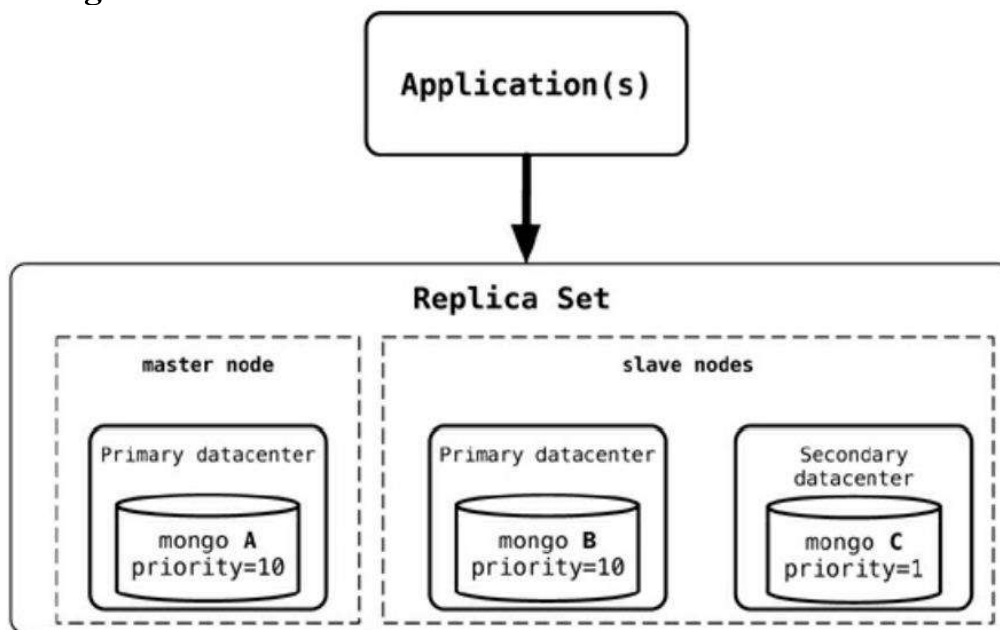
1. Primary Node:
 - Handles all write operations.
 - Only one primary per replica set.
 - Clients can read from the primary.
2. Secondary Nodes:
 - Replicate the data from the primary node.
 - Can serve read operations if configured.
 - If the primary fails, an eligible secondary is elected as the new primary.
3. Arbiter:
 - Does not store data.
 - Participates in elections to break ties but does not become a primary.

Key Features of Replica Sets

1. Data Replication:
 - The primary node records all changes in the oplog (operations log).
 - Secondary nodes replicate these changes asynchronously from the oplog to maintain the same dataset.
2. Automatic Failover:
 - If the primary node becomes unavailable, the replica set members initiate an election process to elect a new primary from the secondary nodes.

- The new primary is chosen based on factors like node priority and freshness of data.
- 3. Read Preference:
 - Clients can specify read preferences to read from the primary or secondary nodes based on their requirements.
 - Read preferences include: primary, primaryPreferred, secondary, secondaryPreferred, and nearest.
- 4. Data Consistency:
 - By default, MongoDB ensures strong consistency for writes by acknowledging write operations only when the data is written to the primary.
 - Read operations can be configured for eventual consistency by reading from secondary nodes.

Working



The application writes or reads from the primary (master) node. When connection is established, the application only needs to connect to one node (primary or not, does not matter) in the replica set, and the rest of the nodes are discovered automatically. When the primary node goes down, the driver talks to the new primary elected by the replica set. The application does not have to manage any of the communication failures or node selection criteria. Using replica sets gives you the ability to have a highly available document data store.

Scaling

Scaling in MongoDB is a critical process that ensures a database can handle increasing data volumes, user traffic and processing demands. As applications grow, maintaining optimal performance and resource utilization becomes essential.

The need for scaling in MongoDB arises due to below factors:

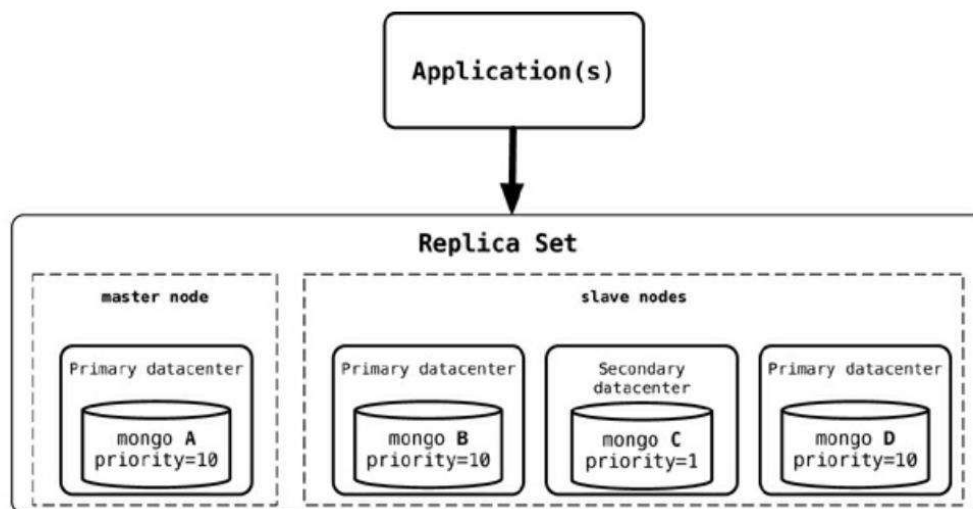
- **Increased Data Volume:** As the amount of data stored in a MongoDB database grows the performance of queries and operations can be affected. Scaling allows to distribution of this data across various servers or nodes and prevents poor performance.

- **Rising User Traffic:** Growing applications often experience a lot of users and increased concurrent requests. Scaling ensures the database can handle a large number of read and write operations and also maintains responsiveness and user experience.
- **Diverse Workloads:** Applications may implement various methods to support new features or functionalities which lead to diverse workloads. Scaling allows organizations to adapt to changing demands by optimizing the database system for different types of queries and operations.
- **Improved Fault Tolerance:** Horizontal scaling in MongoDB is achieved through sharding and enhances **fault tolerance**. By distributing data across multiple servers (**shards**), the system can continue to operate even if one server fails. It ensures **high availability** and **reliability**.
- **Cost-Efficiency:** **Horizontal scaling**, which involves adding more hardware as needed and offers a **cost-effective** approach to handling increased workloads. Organizations can scale incrementally based on **demand**, optimizing infrastructure costs.

Working

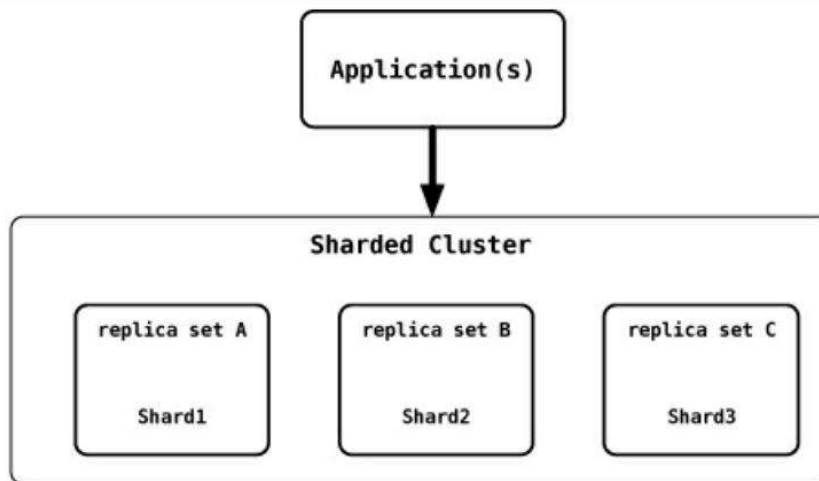
Scaling for heavy read-applications:

Scaling for heavy-read loads can be achieved by adding more read slaves, be directed to the slaves. Given a heavy-read application, with our 3-node replica-set cluster, we can add more read capacity to the cluster as the read load increases just by adding more slave nodes to the replica set to execute reads.



Scaling for write-only applications:

When a new node is added, it will sync up with the existing nodes, join the replica set as secondary node, and start serving read requests. An advantage of this setup is that we do not have to restart any other nodes, and there is no downtime for the application either.



The shard key plays an important role. You may want to place your MongoDB database shards closer to their users, so sharding based on user location may be a good idea. When sharding by customer location, all user data for the East Coast of the USA is in the shards that are served from the East Coast, and all user data for the West Coast is in the shards that are on the West Coast.

Suitable use cases

-  **Event logging-** MongoDB can be effectively used for event logging by leveraging its flexible schema, scalability, and real-time capabilities. Events can be logged directly into MongoDB collections, taking advantage of features like capped collections to manage fixed-size log data efficiently. The database's ability to handle high write loads and its powerful querying capabilities make it suitable for storing and retrieving log events quickly. Additionally, MongoDB can be integrated with log management and analysis tools like the ELK stack or Graylog to enhance log aggregation, search, and visualization, providing a robust solution for real-time event monitoring, debugging, and analysis.
-  **Content Management Systems, Blogging Platforms-** MongoDB is an excellent choice for building content management systems (CMS) and blogging platforms due to its flexible schema, scalability, and ease of handling diverse data types. Its document-oriented structure allows for dynamic content storage, enabling the easy management of various content types like blog posts, comments, and user profiles. MongoDB's scalability ensures that the CMS or blogging platform can handle growing traffic and content volume seamlessly. Additionally, its powerful querying and indexing capabilities facilitate fast retrieval of content, making it suitable for real-time content updates and interactive user experiences. Integration with modern web frameworks and APIs further enhances MongoDB's utility in developing robust and efficient CMS and blogging platforms.
-  **Web Analytics or Real-Time Analytics -** MongoDB is well-suited for web analytics and real-time analytics applications due to its high write throughput, flexible schema design, and powerful aggregation framework. Its ability to ingest and store large volumes of unstructured and semi-structured data quickly makes it ideal for capturing web activity logs, user interactions, and other analytical data points. MongoDB's real-time capabilities allow for immediate processing and analysis of incoming data,

providing timely insights and enabling dynamic dashboards and reports. The database's scalability ensures that it can handle the growing data needs of analytics platforms, while its integration with tools like Apache Kafka and Spark enhances its ability to support complex data processing and analysis pipelines, making it a robust choice for real-time and web analytics solutions.

- ✚ **E-Commerce Applications** - MongoDB is highly effective for e-commerce applications due to its flexible schema, which allows for dynamic and diverse product catalogs that can easily evolve with business needs. Its ability to handle large volumes of transactions with high availability and horizontal scalability ensures that the application can support growing user bases and increased traffic during peak times, such as sales events. MongoDB's powerful indexing and aggregation capabilities enable fast and efficient querying, crucial for providing real-time inventory updates, personalized recommendations, and advanced search features. Additionally, its document-oriented data model aligns well with the complex data structures inherent in e-commerce, such as customer profiles, order histories, and shopping carts, making MongoDB a robust and versatile choice for modern e-commerce platforms.

When not to use (Limitations)

- ✚ **Complex Transactions Spanning Different Operations** -MongoDB is not well-suited for complex transactions that span multiple operations across different collections or documents due to its limited support for multi-document ACID transactions. While MongoDB does offer support for multi-document transactions, it is not as robust as traditional relational databases when dealing with complex transactional workflows that require strict consistency and rollbacks across various entities. This limitation can pose challenges in scenarios requiring intricate financial transactions, inventory management, or other critical operations that demand guaranteed atomicity, consistency, isolation, and durability (ACID) across numerous operations. For such use cases, a relational database might be a more appropriate choice.
- ✚ **Queries against Varying Aggregate Structure-** MongoDB may not be the best choice for scenarios requiring complex queries against varying aggregate structures due to its document-oriented nature. While MongoDB supports aggregation through its powerful aggregation framework, the flexibility of its schema can lead to inconsistent data structures across documents, making it challenging to perform efficient queries or joins on heterogeneous data. In cases where the data structure varies significantly between records or requires frequent schema changes, querying and aggregating data can become inefficient and complex. Relational databases or specialized analytics platforms may be more suitable for handling complex queries against data with varying aggregate structures, as they provide a more rigid and standardized schema for optimized querying and aggregation.